Configure a Database Connection

Configure a Data Source Reference Connection

Anypoint Connector for Database (Database Connector) can connect to any database for which a JDBC driver is available. The following examples show how to connect to the most popular databases such as Derby, Microsoft SQL Server, MySQL, Oracle, and a generic database. Additionally, for advanced uses cases, there are examples showing how to connect to a global data source, configure a JDBC driver, configure connection pooling, and connect to an Oracle database with TNS, both in Anypoint Studio and the XML editor.

Configure a Data Source Reference Connection

Query a Database

Query a Single Record from a Database

Insert, Update, and

Database Connector generates a JDBC data source by using the parameter values you provide when you configure the **Generic Connection** field. If you want to create your own data source and use it to connect to a database, configure the **Data Source Reference Connection** field.

The following example shows how to use Anypoint Studio to configure a data source reference connection to a Derby database:

1. In Studio, in the `src/main/resources` folder, create a `datasource-config.xml` bean file to define a custom `enhydra` data source that connects to a Derby database:

   datasource-config.xml File

   XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="
               http://www.springframework.org/schema/beans http://www.springframewo
<bean id="jdbcDataSource" class="org.enhydra.jdbc.standard.StandardDataSource"
 destroy-method="shutdown">
   <property name="driverName" value="org.apache.derby.jdbc.EmbeddedDriver"/>
   <property name="url" value="jdbc:derby:muleEmbeddedDB;create=true"/>
</bean>
</beans>
```

2. Return to your Mule flow, go to the **Global Elements** view, and click **Create** to open the **Choose Global Type** window.
3. In the **Filter** field, type `spring`, select **Spring Config**, and click **OK**.
4. In the **Spring Config** window **Files** field, add `datasource-config.xml` to reference the spring configuration file with the Spring module.
5. Click **OK**.

The following screenshot shows the configuration:

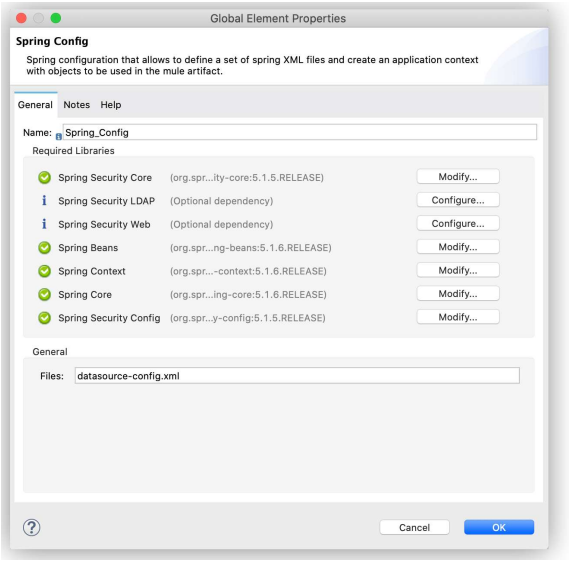


Figure 1. Spring Configuration Data Source

6. In the **Global Elements** view, click **Create** to open the **Choose Global Type** window.
7. In the **Filter** field, type `database`, select **Database Config**, and click **OK**.
8. In the **Database Config** window, for the **Connection** field, select **Data Source Reference Connection**.
9. Click **Configure** to add the required JDBC driver and select either:
    - **Use local file**
      Install the library using a local file.
    - **Add Maven dependency**
      Install a Maven dependency to add to the project.
10. For **Data source ref**, add the bean ID `jdbcDataSource` created in the `datasource-config.xml` bean file.
11. Click **Test Connection** to confirm that Mule can connect to the database.

12. Click **OK**.

The following screenshot shows the configuration:

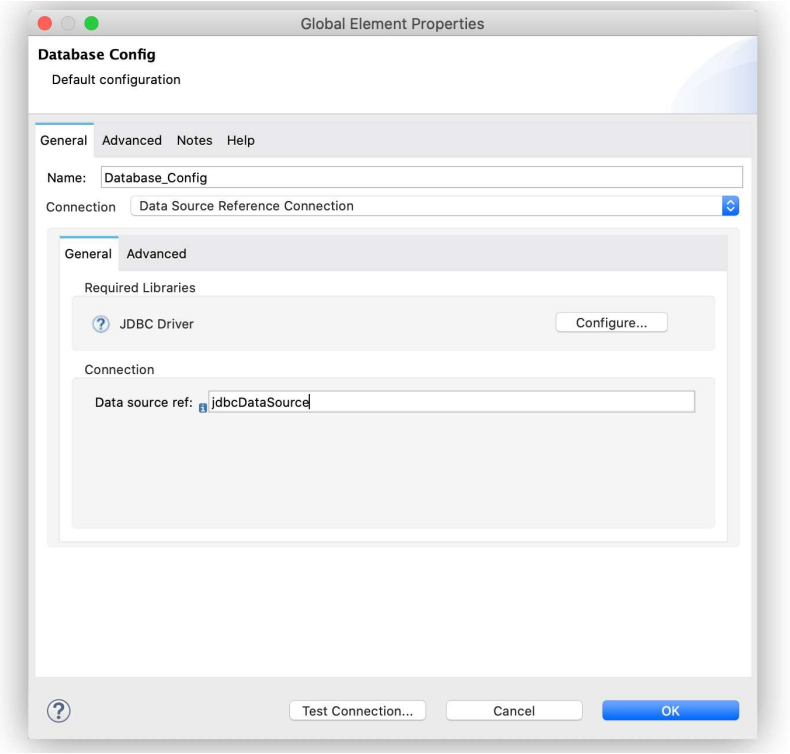


Figure 2. Data Source Reference Connection

In the XML editor, both the spring and database configurations look like this:

```xml
<spring:config name="datasource" files="datasource-config.xml" />

<db:config name="Database_Config">
   <db:data-source-connection dataSourceRef="jdbcDataSource" />
</db:config>
```

Configure a Data Source Reference Connection When in XA Transaction Mode

If your Mule app requires operations bound to XA transactions, and connection pooling is enabled for the database configuration in use, the default connection pool implementation for Data Source Reference Connection is Enhydra XAPool.

Enhydra XAPool does not support prepared statements. To execute these operations use the Bitronix connection pool. Refer to Configuring the Bitronix Transaction Manager documentation for details.

> NOTE
> When using XA transactions, you must provide an `XADataSource` object in the JDBC data source spring bean configuration file of your app.

The following example shows how to configure a **Data Source Reference Connection** for a Derby database when in XA transaction mode:

1. In Studio, in the `src/main/resources` folder, create a `datasource-config.xml` spring bean file.
2. Add the `XADataSource` object to the end of the class `org.enhydra.jdbc.standard.Standard`, for example, `class="org.enhydra.jdbc.standard.StandardXaDataSource`:

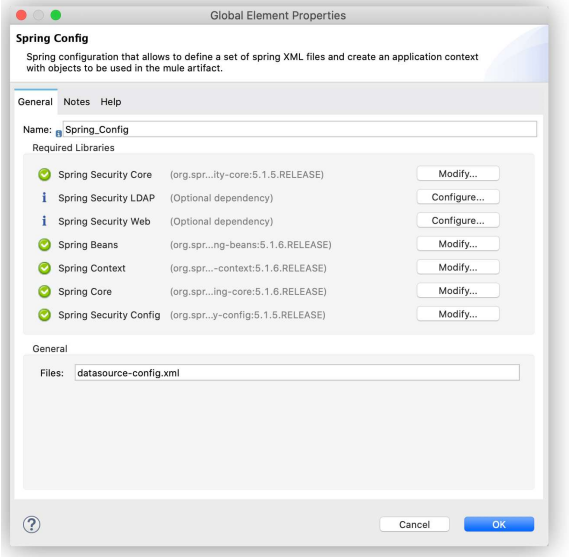
   datasource-config.xml File

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="
               http://www.springframework.org/schema/beans http://www.springframewo

    <bean id="jdbcDataSource" class="org.enhydra.jdbc.standard.StandardXaDataSource
        <property name="driverName" value="org.apache.derby.jdbc.EmbeddedDriver"/>
        <property name="url" value="jdbc:derby:muleEmbeddedDB;create=true"/>
    </bean>

</beans>
```

3. Return to your Mule flow and navigate to **Global Elements** > **Create** to open the **Choose Global Type** window.
4. In **Filter**, type `spring`, select **Spring Config**, and click **OK**.
5. In **Spring Config** > **Files**, add `datasource-config.xml` to reference the Spring configuration file with the Spring module.
6. Click **OK**.



8. In **Global Elements**, click **Create** to open the **Choose Global Type** window.
9. In **Filter**, type `database`, select **Database Config**, and click **OK**.
10. In **Database Config** > **Connection**, select **Data Source Reference Connection**.
11. Click **Configure** to add the required JDBC driver and select either:
    - **Use local file**
      Install the library using a local file.
    - **Add Maven dependency**
      Install a Maven dependency to add to the project.
12. For **Data source ref**, add the bean ID `jdbcDataSource` created in the `datasource-config.xml` bean file.
13. In the **Advanced** tab, configure the **Pooling profile**, **Column types**, and **Reconnection** fields.
14. Click **Test Connection** to confirm that Mule can connect to the database.
15. Click **OK**.

In the XML editor, both the Spring and database configurations look like this:

```xml
<spring:config name="spring-beans" files="datasource-spring-beans.xml"/>

<db:config name="db-datasource">
    <db:data-source-connection dataSourceRef="jdbcDataSource">
        <reconnection failsDeployment="true">
            <reconnect frequency="4000" count="4"/>
        </reconnection>
        <db:column-types>
            <db:column-type id="124"
                            typeName="column_type"
                            className="class_name"/>
        </db:column-types>
        <db:pooling-profile maxPoolSize="6" minPoolSize="1"
                            acquireIncrement="2"
                            maxWait="1" maxWaitUnit="MINUTES"/>
    </db:data-source-connection>
</db:config>
```

Configure a Derby Connection

Use the **Derby Connection** field type to connect to a Derby database.

The following example shows how to configure the connection in Anypoint Studio:

1. In Studio, navigate to the **Global Elements** view.
2. Click **Create** to open the **Choose Global Type** view.
3. In the **Filter** field, type `database`, select **Database Config**, and click **OK**.

4. In the **Database Config** window, for the **Connection** field, select **Derby Connection**.
5. Click **Configure** to add the required Derby JDBC driver and select one of:
   - **Add recommended library**
     Install the recommended library.
   - **Use local file**
     Install the library using a local file.
   - **Add Maven dependency**
     Install a Maven dependency to add to the project.
6. Set **Database** to `derbyDB` and **Subsub protocol** to `directory`.
7. On the **Transactions** tab, optionally specify the transaction isolation, or XA transactions when connecting to the database.
8. On the **Advanced** tab, optionally specify connection pooling and reconnection information, including a reconnection strategy.
9. Click **Test Connection** to confirm that Mule can connect to the database.
10. Click **OK**.

The following screenshot shows the configuration in Studio:

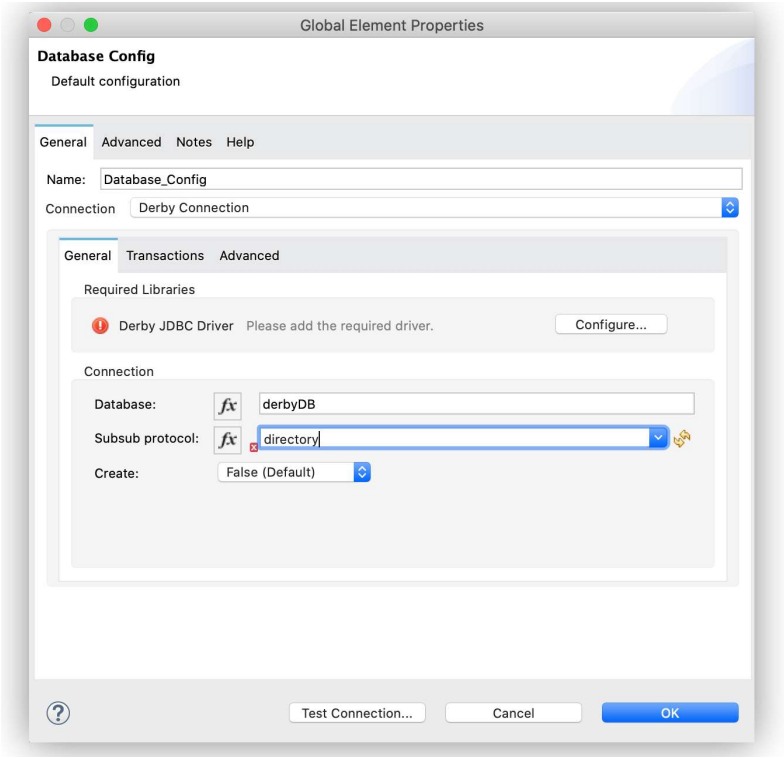


Figure 3. Derby Connection configuration

In the XML editor, the configuration looks like this:

```xml
<db:config name="Database_Config" >
                <db:derby-connection database="derbyDB" />
        </db:config>
```

Configure a Generic Connection

Use the **Generic Connection** field type to connect to:

- A database for which MuleSoft does not provide a specific connection type
- A supported database in which you are using custom options that are not included in the connection types

The following example shows how to connect to an H2 database in Anypoint Studio:

1. In Studio, navigate to the **Global Elements** view.
2. Click **Create** to open the **Choose Global Type** view.
3. In the **Filter** field, type `database`, select **Database Config**,and click **OK**.
4. In the **Database Config** window, for the **Connection** field select **Generic Connection**.
5. Click **Configure** to add the required JDBC driver and select either:
   - **Use local file**
     Install the library using a local file.
   - **Add Maven dependency**
     Install a Maven dependency to add to the project.
6. Set **URL** to `jdbc:h2:file:/tmp/datasenseDBand` and **Driver class name** to `org.h2.Driver`.
7. Click **Test Connection** to validate the connection with the database.
8. Click **OK**.

The following screenshot shows the configuration in Studio:

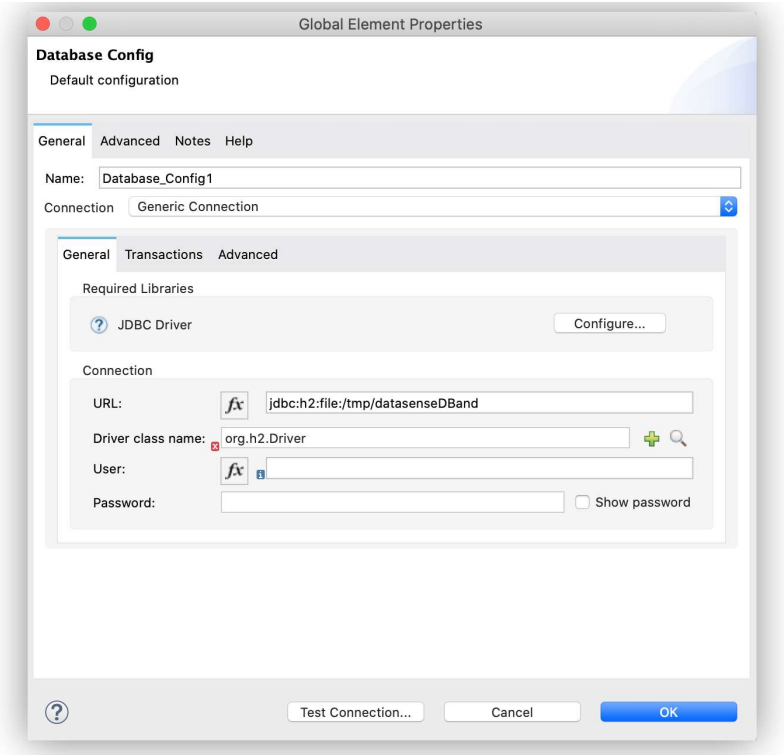


Figure 4. Generic Connection configuration

In the XML editor, the configuration looks like this:

```xml
<db:config name="Database_Config">
    <db:generic-connection driverClassName="org.h2.Driver"
    url="jdbc:h2:file:/tmp/datasenseDB"/>
</db:config>
```

Configure a Microsoft SQL Server Connection

Use the **Microsoft SQL Server Connection** field type to connect to a Microsoft SQL Server database.

The following example shows how to configure the connection in Anypoint Studio:

1. In Studio, navigate to the **Global Elements** view.
2. Click **Create** to open the **Choose Global Type** view.
3. In the **Filter** field, type `database`, select **Database Config**, and click **OK**.
4. In the **Database Config** window, for the **Connection** field, select **Microsoft SQL Server Connection**.
5. Click **Configure** to add the required Microsoft SQL Server driver and select one of:
   - **Add recommended library**
     Install the recommended library.
   - **Use local file**
     Install the library using a local file.
   - **Add Maven dependency**
     Install a Maven dependency to add to the project.
6. Set **Host** to `0.0.0.0`, **Port** to `553`, **User** to `Max`, **Password** to `password` and **Database name** to `microsoftDB`.
7. On the **Transactions** tab, optionally specify the transaction isolation, or XA transactions when connecting to the database.

8. On the **Advanced** tab, optionally specify connection pooling and reconnection information, including a reconnection strategy.
9. Click **Test Connection** to confirm that Mule can connect to the database.
10. Click **OK**.

The following screenshot shows the configuration in Studio:

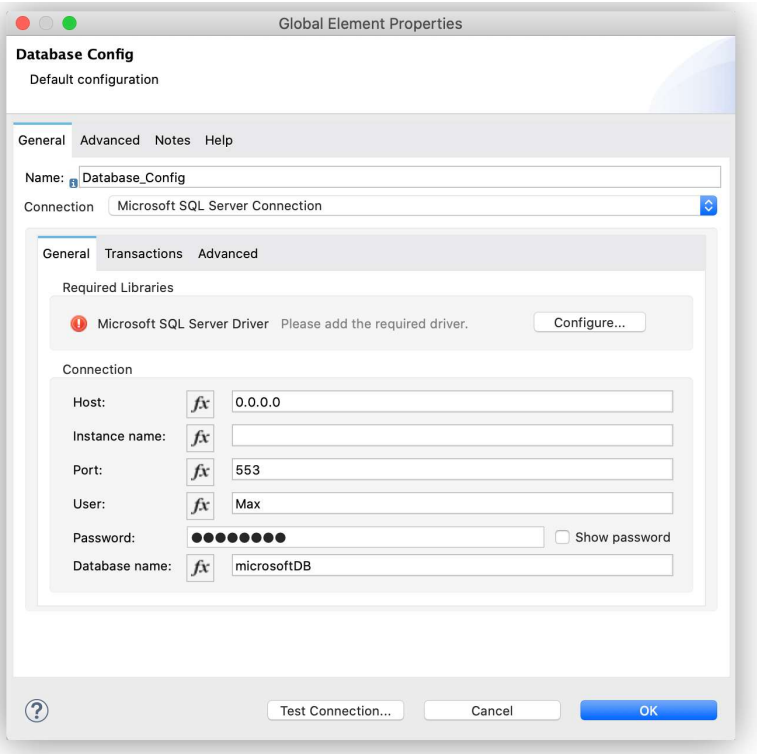


Figure 5. Microsoft SQL Server Connection configuration

In the XML editor, the configuration looks like this:

XML |

```xml
<db:config name="Database_Config">
    <db:mssql-connection
    host="0.0.0.0"
    port="553"
```

```
    user="Max"
    password="password"
    databaseName="microsoftDB" />
</db:config>
```

Configure a MySQL Database Connection

Use the **MySQL Connection** field type to connect to a MySQL database.

The following example shows how to configure the connection in Anypoint Studio:

1. In Studio, navigate to the **Global Elements** view.
2. Click **Create** to open the **Choose Global Type** view.
3. In the **Filter** field, type `database`, select **Database Config**, and click **OK**.
4. In the **Database Config** window, for the **Connection** field, select **MySQL Connection**.
5. Click **Configure** to add the required MySQL JDBC driver and select one of:
   - **Add recommended library**
     Install the recommended library.
   - **Use local file**
     Install the library using a local file.
   - **Add Maven dependency**
     Install a Maven dependency to add to the project.
6. Set **Host** to `0.0.0.0`, **Port** to `3306`, **User** to `Max`, **Password** to `password`, and **Database** to `someSchema`.

7. On the **Transactions** tab, optionally specify the transaction isolation, or XA transactions when connecting to the database.
8. On the **Advanced** tab, optionally specify connection pooling and reconnection information, including a reconnection strategy.
9. Click **Test Connection** to confirm that Mule can connect to the database.
10. Click **OK**.

The following screenshot shows the configuration in Studio:

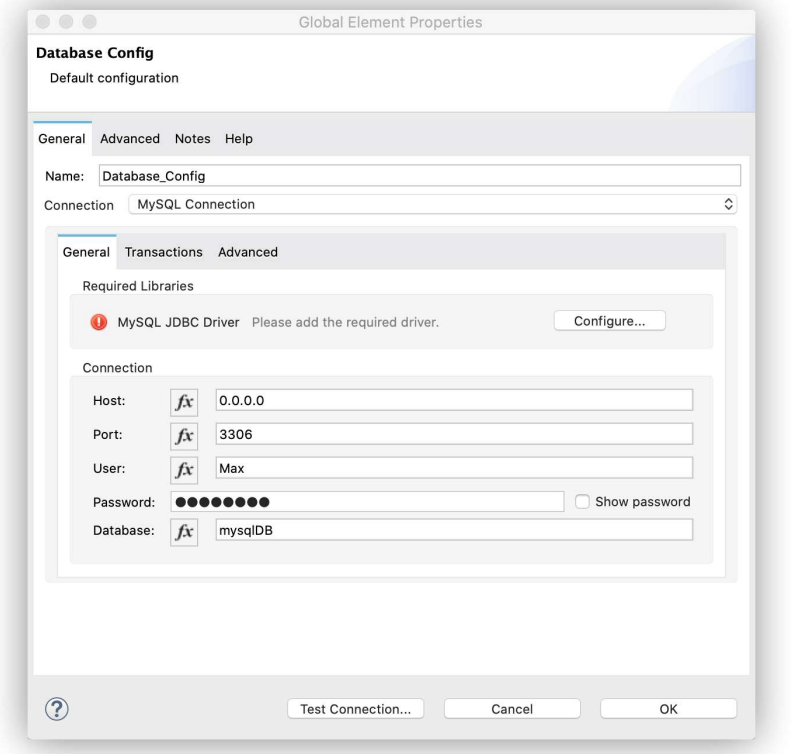


Figure 6. MySQL Connection configuration

In the XML editor, the configuration looks like this:

XML |

```
<db:config name="Database_Config">
  <db:my-sql-connection
```

```
  host="0.0.0.0"
  port="3306"
  user="Max"
  password="POWERS"
  database="mysqlDB"/>
</db:config>
```

Configure an Oracle Connection

Use the **Oracle Connection** field type to connect to an Oracle database.

The following example shows how to configure the connection in Anypoint Studio:

1. In Studio, navigate to the **Global Elements** view.
2. Click **Create** to open the **Choose Global Type** view.
3. In the **Filter** field, type `database`, select **Database Config** and click **OK**.
4. In the **Database Config** window, for the **Connection** field, select **Oracle Connection**.
5. Click **Configure** to add the required Oracle JDBC driver and select either:
   - **Use local file**
     Install the library using a local file.
   - **Add Maven dependency**
     Install a Maven dependency to add to the project.
6. Set **Host** to `192.168.99.100`, **Port** to `1521`, **User** to `system`, **Password** to `oracle`, and **Instance** to `xe`.
7. On the **Transactions** tab, optionally specify the transaction isolation, or XA transactions when connecting to the database.

8. On the **Advanced** tab, optionally specify connection pooling and reconnection information, including a reconnection strategy.
9. Click **Test Connection** to confirm that Mule can connect to the database.
10. Click **OK**.

The following screenshot shows the configuration in Studio:

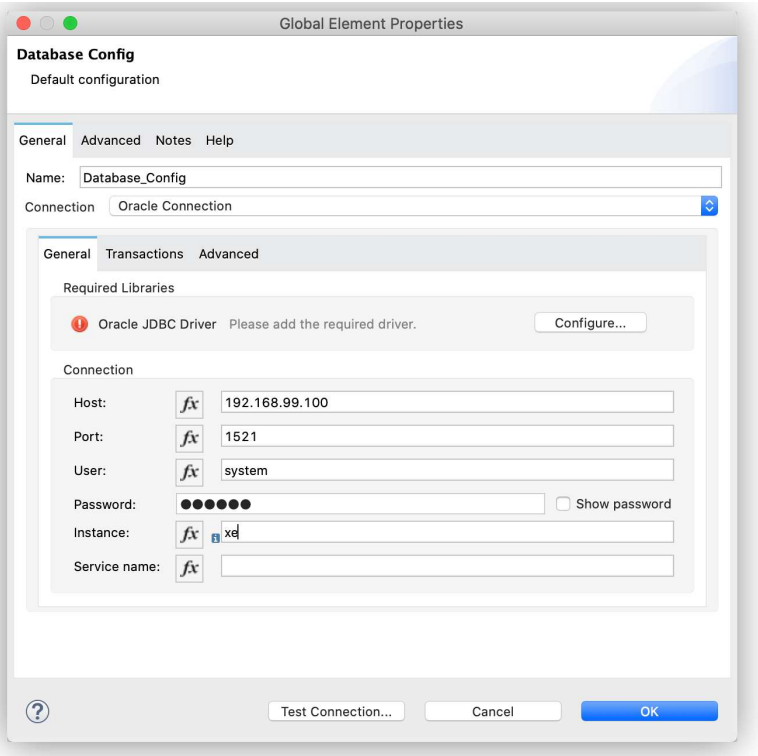


Figure 7. Oracle Connection configuration

In the XML editor, the configuration looks like this:

XML |

```xml
<db:config name="Database_Config">
  <db:oracle-connection
  host="192.168.99.100"
  port="1521" instance="xe"
```

```
  user="system"
  password="oracle" />
</db:config>
```

Configure the JDBC Driver

After you configure the connections, add the JDBC driver. The following example shows how to add a driver for the MySQL database:

1. In Studio, add the driver using either of the following options:
   - In the **Database Config** window, click **Configure**, select **Add Maven dependency**, and pick the dependency:

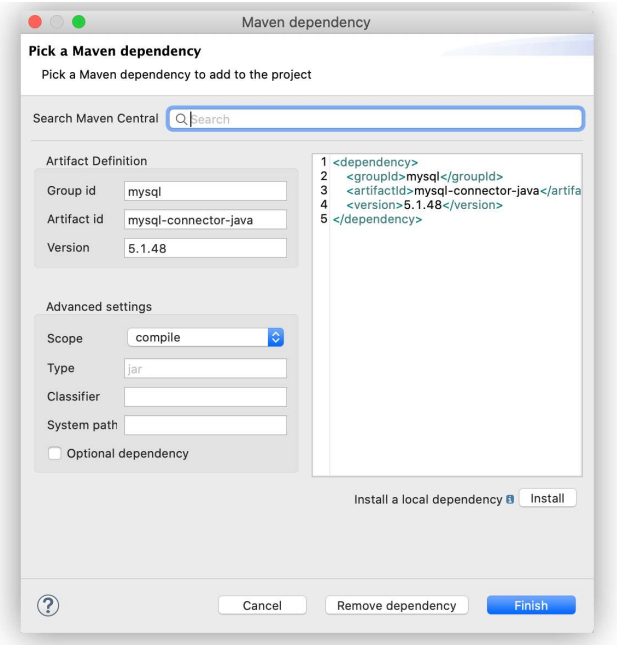


Figure 8. Pick Maven dependency

- Add the dependency directly to your project's `pom.xml` file:

```xml
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>5.1.48</version>
</dependency>
```

2. Configure your `pom.xml` file so that Mule exposes the driver dependency to Database Connector:

```xml
<build>
    <plugins>
        <!-- Only used to declare the shared libraries-->
        <plugin>
            <groupId>org.mule.tools.maven</groupId>
            <artifactId>mule-maven-plugin</artifactId>
            <version>1.0.0</version>
            <configuration>
                <sharedLibraries>
                    <sharedLibrary>
                        <groupId>mysql</groupId>
                        <artifactId>mysql-connector-java</artifactId>
                    </sharedLibrary>
                </sharedLibraries>
            </configuration>
        </plugin>
    </plugins>
</build>
```

Configure Connection Pools

Establishing connections is costly in terms of connectors, the relational database management system (RDBMS) network, and processing overhead. Database Connector supports the use of pooling profiles that enable you to build a connection pool. This is a group of established connections that the connector can use to perform required operations without having to establish new connections every time.

When you configure the **Pooling profile** field in the global element of your application, deselect the **Test connection on checkout** field to improve the connector performance with connection pooling. If you leave the field selected as default (`TRUE`), every time a connection is checked out of the pool, the connection pool performs connectivity checks against the database, which increases the time required for each operation to complete.

All connection type configurations accept the **Pooling profile** field configuration. In JDBC, pooling occurs at the data source level, so to enable pooling, you have to configure the global data source.

The following example shows how to configure a pooling profile for an Oracle database connection with the **Test connection on checkout** field set to `FALSE`:

1. In the **Database Config** window, in the **Advanced** tab, for **Pooling profile** select **Edit inline**.
2. Set the following fields:
   - **Max pool size**: `30`
   - **Min pool size**: `4`
   - **Acquire increment**: `1`

- **Prepared statement cache size** `5`
- **Max wait**: `0`
- **Max wait unit**: `SECONDS (Default)`
- **Max idle time** `60`
- **Max statements** `0`

3. Deselect **Test connection on checkout**.
   This sets the field as `FALSE`.
4. For **Column types** select **Edit inline**.
5. Click the (+) plus sign to add a column type.
6. In the **Column type** window, set **Id** to `2003`, **Type name** to `1000`, and click **Finish**.
7. Repeat step 5.
8. To add a new column type, set **Id** to `2002`, **Type name** to `PHONE_NUMBER_ARRAY`, and click **Finish**.

The following screenshot shows the configuration in Studio:

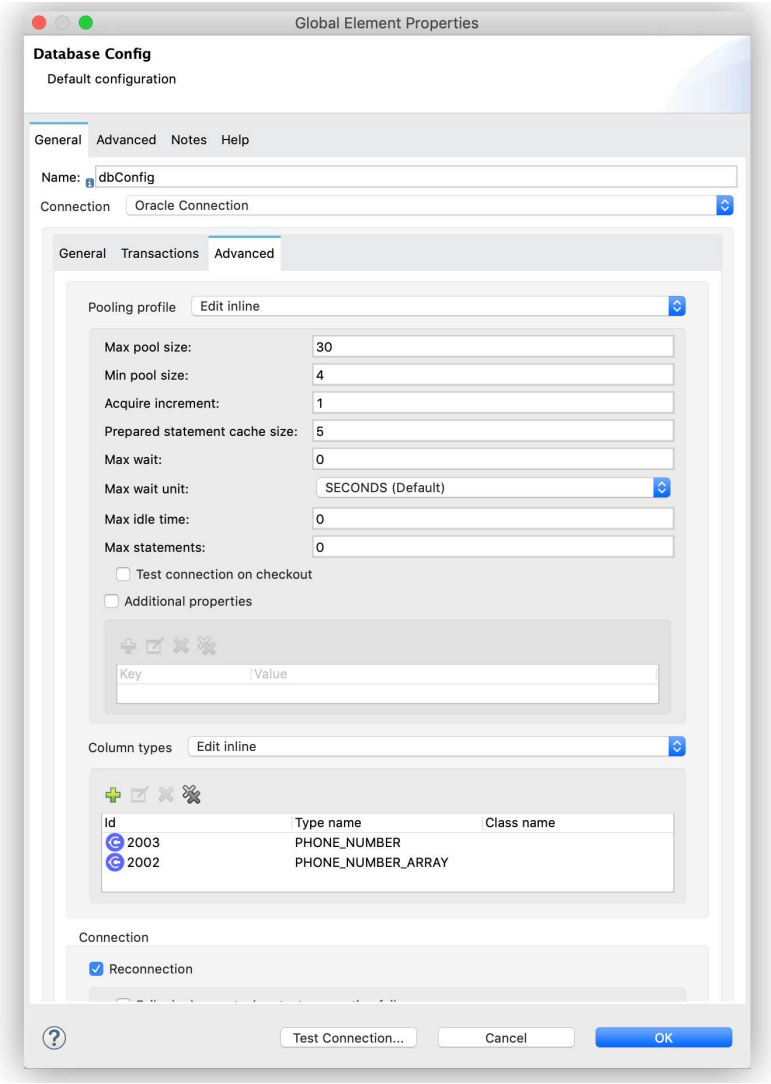


Figure 9. Pooling Profile Configuration

In the XML editor, the `<db:pooling-profile>` and `testConnectionOnCheckout="false"` configurations look like these:

```xml
<db:config name="dbConfig">
                <db:oracle-connection host="localhost" user="myuser" password="mypass
                        <db:pooling-profile minPoolSize="4" maxPoolSize="30" testConn
```

```
                <db:column-types>
                    <db:column-type id="2003" typeName="PHONE_NUMBER" />
                    <db:column-type id="2002" typeName="PHONE_NUMBER_ARRA
                </db:column-types>
            </db:oracle-connection>
        </db:config>
```

Configure an Oracle Connection with TNS Names

The following example shows how to configure the **Generic Connection** field to connect to an Oracle database that relies on the `tnsnames.ora` file. If a firewall separates your Mule runtime engine from the database, use the `ENABLE=BROKEN` parameter to enable a keep-alive connection at the TNS connection. If a keep-alive connection is not set, the connection can drop traffic due to expiring inactivity timers.

1. In Studio, navigate to the **Global Elements** view.
2. Click **Create** to open the **Choose Global Type** view.
3. In the **Filter** field, type `database`, select **Database Config** and click **OK**.
4. In the **Database Config** window, for the **Connection** field, select **Generic Connection**.
5. Click **Configure** to add the required JDBC driver and select either:

    - **Use local file**
      Install the library using a local file.

- **Add Maven dependency**
  Install a Maven dependency to add to the project.

6. Set the connection **URL** to
   `jdbc:oracle:thin:${oracle.user}/${oracle.password}@(DESCRIPTION=(ENABLE=BROKEN)(FAILOVER=ON)(ADDRESS_LIST=(ADDRESS=(PROTOCOL=TCP)(HOST=${oracle.host1})(PORT=${oracle.port}))(ADDRESS=(PROTOCOL=TCP)(HOST=${oracle.host2})(PORT=${oracle.port})))(CONNECT_DATA=(SERVICE_NAME=${oracle.service})))`.
7. Set **Driver class name** to `oracle.jdbc.driver.OracleDriver`.
8. Click **Test Connection** to validate the connection with the database.
9. Click **OK**.

The following screenshot shows the configuration in Studio:

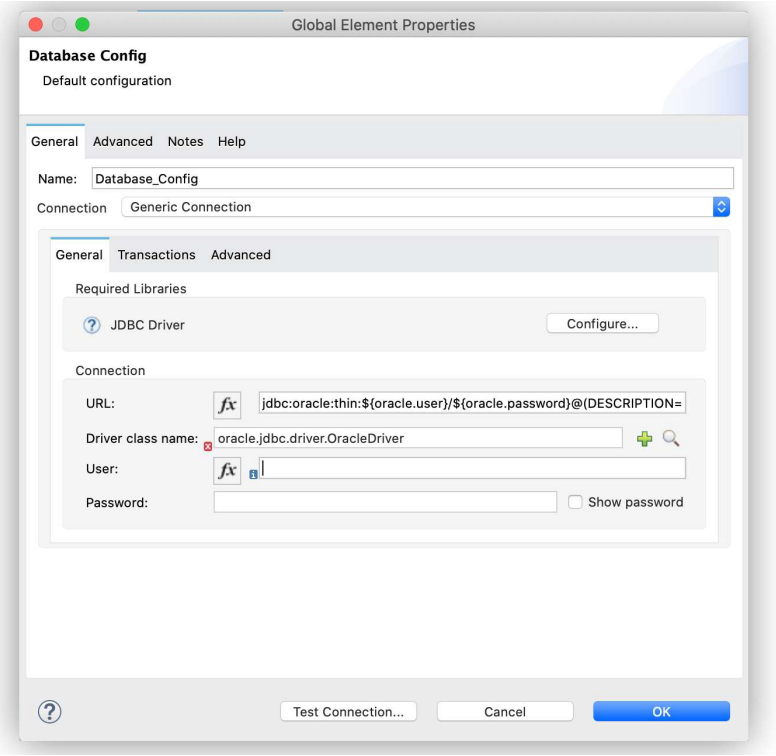


Figure 10. Oracle with TNS configuration

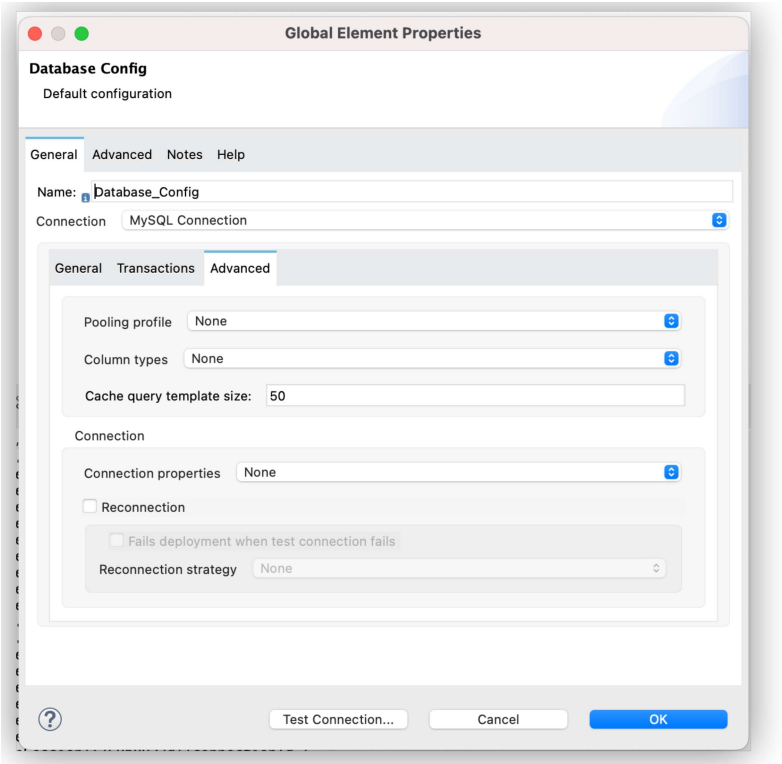
In the XML editor, the configuration looks like this:

```xml
<db:config name="Database_Config">
    <db:generic-connection driverClassName="oracle.jdbc.driver.OracleDriver"
    url="jdbc:oracle:thin:${oracle.user}/${oracle.password}@(DESCRIPTION=(ENABLE=BROK
</db:config>
```

Configure Cache Query Template Size

The following example shows how to configure the **Cache Query Template Size** field for the connection.

1. In Studio, open the Database Global Configuration window.
2. Select the **Advanced** tab.
3. Set the **Cache Query Template Size** field to your desired value.



See Also

- Database Connector Reference
- Salesforce Help

Back to top

DID THIS ARTICLE SOLVE YOUR ISSUE?
Let us know so we can improve!

Yes No

Legal | Terms of Service | Privacy | Trust | Contact | Responsible Disclosure | Cookie Settings
Your Privacy Choices

© Copyright 2026 Salesforce, Inc. All rights reserved. Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States